



Türkçe Encoder Tasarımı – MoganAI

Furkan Yılmaz

Habibe Aleyna Taşdemir

Muhammed Faruk Gözay

Bekir Top

İçindekiler

Abstract	3
1 Giriş ve Motivasyon	3
2 İlgili Çalışmalar ve Konumlandırma	3
3 Veri Toplama	4
3.1 Kaynak seçimi ve erişim kısıtları	4
3.2 İndeksleme	4
3.3 İndirme	4
4 Çıkarım ve Ön Filtreleme	4
5 Anlam-Bazlı Kalite Filtresi: Üç Yaklaşımın Karşılaştırmalı Tarihi	5
5.1 Deneme 1 — Sıfırdan/pretrained fastText (baseline)	5
5.2 Deneme 2 — LLM-etiketli veriyle fastText (FineWeb-Edu yöntemi)	5
5.3 Deneme 3 — Türkçe BERT fine-tuning	5
5.4 Dönüm Noktası — BERT Kararlarının fastText'e Damıtılması	6
5.5 Yorum: Neden Doğrudan Sınıf-3 (LLM→fastText) Değil de Bu Sıra?	6
6 Üretim Mimarisine Entegrasyon	6
6.1 Kaynak/worker optimizasyonu	6
7 Deduplication	6
8 Üretim Ölçeği ve Kaynak Kullanımı	7
9 Sınırlılıklar	7
10 Basılı Kaynaklardan Metin Çıkarımı: OCR Model Seçiminin Deneysel Tarihi	7
10.1 Sorunun iki katmanlı doğası	7
10.2 Deneme 1 — Yerel OCR modelleri	7
10.3 Deneme 2 — Colab üzerinde VLM (Qwen3-VL)	8
10.4 Karar — API tabanlı batch işleme (Gemini 2.5 Flash-Lite)	8
11 Pre-tokenizasyon: Normalizasyon ve Kod Duyarlılığı	9
11.1 Ortak hat	10
11.2 Kod hattı	10
11.3 Tokenizer eğitiminde ölçülen üç problem	12
11.4 Girinti token'ları: allow_whitespace_only_pieces	12
12 Tokenizer Tasarımı: Vocab Boyutu ve Special Token'lar	13
12.1 Vocab boyutu trade-off'u	13
12.2 Special token yerleşimi	15
12.3 HuggingFace dönüşümü ve bir parite hatası	15
13 Masking Stratejisi İçin GPU-suz Ön Analiz	16
14 Pretraining Mimarisi: CLM→MLM Curriculum	17
14.1 Literatür temeli	17
14.2 Mimari: dört sessiz-bug tuzağı	18
14.3 Ablation için bellek ve sayısal kararlar	18
14.4 Ablation Veri hattı	18

15 Ablasyon Sonuçları: CLM→MLM Curriculum vs. Saf MLM	19
15.1 Deney kurulumu	19
15.2 Linear probe: temsil kalitesi	19
15.3 MS MARCO: asıl ayrışma	20
15.4 Neden: embedding geometrisi	21
15.5 Sonuç ve sınırlılıklar	21

Durum: Devam eden çalışma (work in progress). Bu rapor, corpus toplama ve filtreleme sürecinin şu ana kadarki metodolojisini ve tasarım kararlarını belgeler. Modelin kendisinin eğitimi ve downstream değerlendirmesi henüz bu raporun kapsamında değildir.

Abstract

Türkçe için, üçüncü parti çok-dilli derlemelerin (mC4, OSCAR, CulturaX) bir alt kümesi olarak değil, doğrudan Common Crawl ham verisinden ve FineWeb’in temizlenmiş 2013-2024 arşivinden üretilen, bağımsız bir pretraining corpus’u inşa ediyoruz. Katkımız iki kısımdır: (1) Common Crawl’ın aylık anlık görüntülerinden (CC-MAIN-2025-, CC-MAIN-2026-) Türkçe içeriği S3 byte-range erişimiyle çıkaran, tek geçişte indirme+temizleme+kalite filtrelemesi yapan bir üretim hattı; (2) bu hat için, önce büyük dil modeli (LLM) tabanlı etiketleme ve transformer (BERT) tabanlı bir sınıflandırıcıyla başlayıp, o sınıflandırıcının kararlarını milyonlarca örnek üzerinde damıtarak (distillation) elde edilen, ~90 kat daha hızlı ve orijinal BERT ile %94.4 uyumlu bir fastText kalite sınıflandırıcısı. Bu rapor, üç ayrı kalite-filtreleme yaklaşımının (fastText-baseline → LLM-etiketli fastText → BERT → damıtılmış fastText) neden ve nasıl birbirini izlediğini, her birinin ölçülen başarısız olma biçimini ve nihai mimarinin üretim ölçeğindeki performansını raporlar.

1 Giriş ve Motivasyon

Türkçe için kamuya açık, büyük ölçekli pretraining verisi büyük ölçüde çok-dilli derlemelerin (mC4, OSCAR, CC-100, CulturaX) bir alt kümesi olarak var olur; bunlar tek bir dile özel kalite filtrelemesi veya küratörlüğü içermez. Türkçe’ye özel, bağımsız şekilde eğitilmiş (yabancı bir temel modelin fine-tune’u değil) bir foundation model için veri hazırlığının kendisinin ayrı bir mühendislik problemi olduğu — Kumru gibi küçük ekiplerin bunu başarabildiği örneklerden hareketle — bu çalışmanın çıkış noktasıdır.

Kapsam iki tamamlayıcı kaynağa ayrıldı:

2013–2024: HuggingFace’in FineWeb derlemesi (Common Crawl’ın zaten HTML çıkarımı ve temel filtrelemesi yapılmış hâli) üzerinden Türkçe alt-kümenin çekilip kendi kalite filtresinden geçirilmesi.

2025 Ocak – 2026 Nisan: FineWeb’in henüz kapsamadığı güncel dönem için, Common Crawl’ın ham aylık anlık görüntülerinden (CC-MAIN-YYYY-WW) sıfırdan indirme ve tam işleme hattı.

2 İlgili Çalışmalar ve Konumlandırma

Yöntem seçimlerimiz büyük ölçüde CCNet (Wenzek ve ark., 2019) ve FineWeb/FineWeb-2 (HuggingFace, 2024) tarafından yerleşik hâle getirilen desenleri izler: WARC’tan doğrudan metin çıkarımı (WET dosyaları yerine), fastText tabanlı dil kimliklendirme, ve kalite/tekrar filtrelerinin sıralı bir zincir olarak uygulanması. Bu çalışmaların İngilizce/çok-dilli ölçekte kurduğu mimariyi Türkçe’ye uyarlarken karşılaştığımız temel fark, hazır bir Türkçe kalite sınıflandırıcısının bulunmaması ve bunu sıfırdan üretmemiz gerekliliği.

3 Veri Toplama

3.1 Kaynak seçimi ve erişim kısıtları

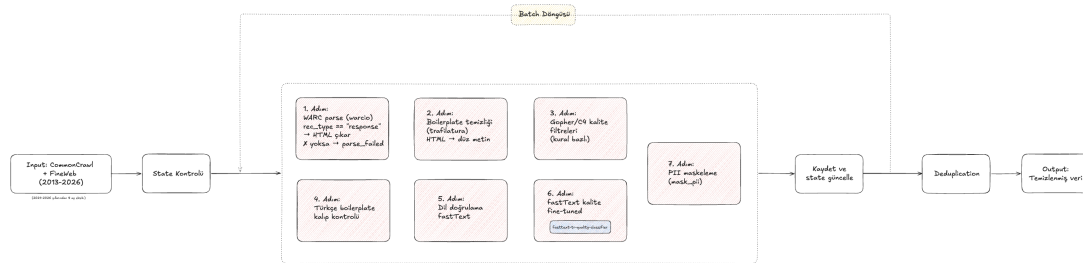
Common Crawl’ın Türkçe içeriğe local (kimlik doğrulamasız) erişimi index seviyesinde kısıtlıdır; pratikte veri yalnızca AWS S3 (s3://commoncrawl, us-east-1) üzerinden, kimliklendirilmiş bir AWS hesabıyla indirilebilir durumdadır. Bu, mimarinin AWS EC2 üzerinde kurulmasının birincil nedenidir.

3.2 İndeksleme

Her aylık crawl için, o ayın columnar index’i (Parquet, Athena/DuckDB ile sorgulanabilir) üzerinden content_languages alanına göre Türkçe içerik önce indekslenir (queries/build_tur_index.py), tüm WARC dosyaları indirilmeden önce. Bu adım warc_filename, warc_record_offset, warc_record_length üçlüsünü çıkarır ve sonucu lokal bir Parquet dosyasına yazar; 300 shard’lık bir crawl index’i tipik olarak tamamlanır (kesinti durumunda –resume ile kaldığı shard’dan devam eder).

3.3 İndirme

İndeksten elde edilen offset/length bilgisiyle, her Türkçe kayıt HTTP Range GET ile WARC dosyasının yalnızca ilgili byte aralığı indirilerek çekilir — dosyanın tamamı indirilmez. Üretim ölçeğinde ölçülen indirme hızı ~450–480 kayıt/saniye (64 eşzamanlı worker, c7i.8xlarge, 32 vCPU).



Şekil 1: Toplama ve işleme hattı: CommonCrawl + FineWeb girdisinden 7 adımlık işleme zincirine ve deduplication çıktısına.

Batch işleme döngüsü: CommonCrawl + FineWeb girdisi, state kontrolü, 7 adımlık işleme zinciri (WARC parse, boilerplate temizliği, Gopher/C4 kalite filtreleri, Türkçe boilerplate kontrolü, dil doğrulama, fastText kalite sınıflandırması, PII maskelama), kayıt/state güncelleme, deduplication, çıktı.

4 Çıkarım ve Ön Filtreleme

WARC kayıtları warcio ile ayrıştırılır, HTML gövdesinden ana metin trafilatūra ile çıkarılır (boilerplate/menü/reklam metninin ayıklanması için WET’in hazır çıkarımı yerine bilinçli olarak tercih edilmiştir — FineWeb/CCNet’in izlediği aynı yaklaşım). Metin, fastText dil

kimliklendirme modeli (lid.176.ftz) ile yeniden doğrulanır (Common Crawl’ın HTML-seviyesi content_languages etiketine güvenilmez). Ardından Gopher/C4 tarzı sezgisel kalite filtreleri (uzunluk, sembol oranı, tekrar oranı) ve PII maskeleye (TC Kimlik No checksum doğrulaması, telefon, e-posta, IBAN, kredi kartı Luhn algoritması) uygulanır.

5 Anlam-Bazlı Kalite Filtresi: Üç Yaklaşımın Karşılaştırmalı Tarihi

Yukarıdaki sezgisel filtreler, spam/bahis/reklam/SEO içeriğini yeterince ayıkamaz — bu içerik genelde dilbilgisel olarak “geçerli” Türkçe metindir. Bu bölüm, bu boşluğu kapatmak için sırayla denenen üç yaklaşımı, her birinin gözlemlenen başarısızlık biçimiyle birlikte belgeler.

5.1 Deneme 1 — Sıfırdan/pretrained fastText (baseline)

Herhangi bir etiketli veri olmadan, hazır Türkçe kelime vektörleriyle eğitilen bir fastText sınıflandırıcı ilk denemede kullanıldı. Gözlemlenen başarısızlık biçimi: sınıflandırıcı veriyi kör biçimde eldi — kısa/orta uzunluktaki meşru içeriği (Wikipedia tarzı özetler, akademik metin) sistematik olarak düşük kalite olarak işaretleme eğilimi gösterdi. Anlam ayrımı yapmıyor, yüzeysel kelime istatistiklerine aşırı uyum sağlıyordu.

5.2 Deneme 2 — LLM-etiketli veriyle fastText (FineWeb-Edu yöntemi)

FineWeb-Edu’nun izlediği yönteme paralel olarak, ~6000 doküman bir LLM’e (başlangıçta Gemini; günlük/dakikalık kota kısıtları nedeniyle Mistral’e geçildi) 0–5 arası bir kalite/eğitcilik rubriğiyle puanlatıldı (bahis/casino/escort/pornografi içeriği sabit 0; reklam/emlak ilanı/SEO içeriği 3 puanın altında kalacak şekilde rubrik kalibre edildi). Bu ~6000 etiketle fastText yeniden eğitildi. Sonuç: doğrulama doğruluğu ~%54 — üretim için yetersiz, rastgele tahminden çok az iyi. Değerlendirmemiz: 6000 örnek, fastText’in n-gram temelli temsil gücüyle 6 sınıflı bir ayrımı öğrenmesi için yetersiz hacimdeydi.

5.3 Deneme 3 — Türkçe BERT fine-tuning

Aynı ~6000 etiketli örnekle (scored_samples.jsonl, sınıf başına maks. 1000 ile hafif dengelenmiş), Türkçe bir BERT temel modeli fine-tune edildi (train_bert_classifier.py). Ölçülen doğruluk: %70.7 tam isabet, %93.9 $\pm$ 1 isabet — fastText’in en iyi halinden ~10 puan üstünde. Ancak inference hızı chunk başına ~20 dakika (30 vCPU işleme havuzunda), tüm corpus’u bu hızla işlemenin haftalar süreceği anlamına geliyordu — mimari darboğaz, doğruluk değil, ölçeklenebilirlikti.

keep_labels={3,4,5} eşikleriyle test edildiğinde (1 shard, 116.857 doküman), elenen label=2 dokümanları elle incelendi: büyük çoğunluğu gerçek haber/borsa/tarif/makale içeriğiydi — spam kelime listesi zaten bahis/eskort sitelerini önceki adımda elemiş, “bet”/“fiyat” gibi kelimelerin label=2 içinde geçmesi çoğunlukla yanlış pozitif (örn. “İstanbulspor”, “Betondan”). Bu gözlem sonucunda label=2 de kabul edilen kümeye eklendi (BERT_KEEP_LABELS = {2,3,4,5}), az miktarda ürün/reklam metni riski bilinçli olarak göze alınarak.

5.4 Dönüm Noktası — BERT Kararlarının fastText’e Damıtılması

BERT’in ürettiği include/exclude kararları, işlenen chunk’lar boyunca ayrı bir kümede biriktirildi. Bu birikimin hacmi başta hafife alındı; bir gün sonraki kontrolde etiketlenmiş örnek sayısının 480.000’i (yarım milyona yakın) bulunduğu görüldü. Bu ölçekteki veriyle, ikili (clean/garbage) bir fastText sınıflandırıcısı sıfırdan eğitildi (hiperparametreler: lr=1.0, epoch=25, wordNgrams=2, dim=100, loss=softmax).

Sonuç: BERT’in hiç görmediği bir chunk üzerinde, damıtılmış fastText modeli BERT tahminleriyle %94.4 uyum gösterdi; sentetik/keskin örneklerle yapılan ek testte 8/8 doğru sınıflandırma (çoğu $\geq$ %96.9 güven skoruyla), BERT’in bile net karar veremediği kısa-ama-meşru metinlerde de isabetli. Ölçülen throughput: ~8880 doküman/saniye (fastText) vs. ~96.5 doküman/saniye (BERT) — ~90 kat hızlanma, doğruluk kaybı olmadan (aksine, çok daha büyük eğitim verisi sayesinde BERT’in genellemesini de aştığı gözlemlendi).

5.5 Yorum: Neden Doğrudan Sınıf-3 (LLM→fastText) Değil de Bu Sıra?

Bu üç aşamalı yol (fastText-baseline → LLM-etiketli fastText → BERT → damıtılmış fastText) doğrusal olarak verimsiz görünebilir, ama her adımın kendisi olmadan bir sonraki mümkün olmazdı: LLM-etiketleme adımı (5.2) yetersiz kalsa da, BERT’in fine-tune edileceği ilk etiketli veri kümesini (~6000 örnek) üretti; BERT’in kendisi üretim için çok yavaştı ama etiket üreticisi olarak (~480K ölçekte) kullanılabilir hızdaydı. Yani BERT’in rolü üretim sınıflandırıcısından, bir sonraki sınıflandırıcı için öğretmen modele (teacher model) dönüştü — klasik bilgi damıtımı (knowledge distillation) deseni.

6 Üretim Mimarisine Entegrasyon

Damıtılmış fastText modeli, EC2 üzerinde çalışan ana indirme+işleme betiğine (scripts/aws_download_and_process.py) doğrudan entegre edildi. Önceki mimaride kalite filtrelemesi ayrı bir ikinci geçiş olarak (Drive’a yükle → indir → filtrele → tekrar yükle) yapıyordu; entegrasyon sonrası indirme, HTML/dil/PII temizliği ve kalite filtrelemesi tek geçişte, aynı process içinde tamamlanıp doğrudan temiz JSONL olarak Drive’a yazılıyor. Bu, ikinci round-trip’in indirme+yükleme maliyetini (süre ve bant genişliği) tamamen ortadan kaldırdı.

6.1 Kaynak/worker optimizasyonu

fastText kalite modeli (~2.29GB) her paralel işleme worker’ında ayrı yükleniyor. c7i.8xlarge (32 vCPU, 61GB RAM) üzerinde 30 ve 24 worker’da OOM-kill gözlemlendi; 16 worker’da bellek kullanımı stabil kaldı (~23–30GB). Bu deneysel olarak belirlenen sınır, tüm üretim instance’larında sabit tutuluyor.

7 Deduplication

MinHash tabanlı fuzzy deduplication modülü (preprocess_pipeline/dedup_pipeline.py) yazılmıştır (n-gram=5, 14 bucket, bucket başına 8 hash — datatrove varsayılanlarına yakın). Ay-arası incremental dedup stratejisi (her ayın kendi içinde dedup edilip, sonra birikmiş corpus’a karşı karşılaştırılması) tasarlanmıştır.

8 Üretim Ölçeği ve Kaynak Kullanımı

Şu an üç paralel c7i.8xlarge spot instance, 2025 Ocak–Aralık ve 2026 Ocak–Nisan aylarını sırayla işlemektedir (aylık kuyruk sistemi, `scripts/run_month_queue_*.sh`). Gözlemlenen ortalama işleme hızı ~430–590 kayıt/saniye, kabul oranı işlenen aya göre değişmekle birlikte tipik olarak %29–%55 aralığında (kalan kısım dil doğrulama, kalite filtresi veya parse hatası nedeniyle elenir).

9 Sınırlılıklar

Kalite filtresinin eğitim etiketleri nihayetinde BERT’in kararlarından türetildiği için, BERT’in kendi sistematik önyargıları (örn. §5.3’te düzeltilen `label=2` yanlış pozitifleri gibi) damıtma sürecine miras kalmış olabilir.

fastText kalite modeli ikili (clean/garbage) bir ayrım yapar; FineWeb-Edu tarzı ince taneli (0–5) bir eğiticilik skoru üretmez — bu, ileride bir eğitim-kalitesi ayrımı gerekirse ek bir katman gerektirebilir.

10 Basılı Kaynaklardan Metin Çıkarımı: OCR Model Seçiminin Deneyisel Tarihi

Web kaynaklarından toplanan corpus’un yanına, kitap ve tez gibi basılı kaynakların eklenmesi hedeflendi — bu kaynaklar web metnine göre daha uzun bağlamlı, editöryal denetimden geçmiş ve alan-yoğun (tıp, hukuk, siyaset bilimi) içerik sunar. Ancak PDF’ten metne dönüşüm, beklenenden çok daha fazla iterasyon gerektiren bir adım oldu. Bu bölüm §5’teki filtreleme tarihiyle aynı yapıyı izler: her denemenin gözlemlenen başarısızlık biçimi, bir sonraki adımın gerekçesini üretir.

10.1 Sorunun iki katmanlı doğası

PDF’ten metin çıkarımı, pretraining verisi bağlamında iki ayrı gereksinim içerir:

Transkripsiyon doğruluğu — sayfadaki karakterlerin doğru okunması. Türkçe’ye özgü karakterler (ı/İ, ğ, ş, ç, ö, ü) ve taranmış kitaplardaki düşük kontrast bu katmanı zorlaştırır.

Yapısal sınıflandırma — okunan metnin ne olduğunun belirlenmesi. Kapak sayfası, telif hakkı bilgisi, çeviri editörleri listesi, önsöz ve içindekiler sayfası kitabın gerçek içeriği değildir ve corpus’a girmemelidir. Bölüm başlıklarının paragraf metninden ayrılması ise chunk sınırlarının anlamlı yerlerden geçmesini sağlar.

Denenen araçların çoğu birinci katmanı çözer, ikincisini çözmez — bu ayrım model seçiminin belirleyici kriteri oldu.

10.2 Deneme 1 — Yerel OCR modelleri

İlk yaklaşım corpus’u yerel donanımda işlemekti. Denenen araçlar ve gözlemlenen sonuçlar:

Araç	Yaklaşım	Gözlemlenen sonuç
Marker	PDF→Markdown, layout-aware	Kabul edilebilir metin kalitesi, ancak sayfa başına işlem süresi yüksek
PaddleOCR	Klasik OCR (detection + recognition)	Yapısal bilgi üretmiyor; düz metin akışı
baidu/Unlimited-OCR	Layout + OCR birleşik	Başlık tipini görsel boyuta göre veriyor — semantik ayırım yok
Qwen3-VL-4B-Instruct	Genel amaçlı vision-language	Hem transkripsiyon hem semantik sınıflandırma yapabiliyor

Unlimited-OCR’ın başarısızlık biçimi öğreticiydi: model bir metin bloğunun “title” olup olmadığına punto büyüklüğü ve konumdan karar veriyordu. Bu, kapak sayfasındaki yayımevi adını da gerçek bölüm başlığını da aynı kategoriye yerleştiriyordu — yani §10.1’deki ikinci katman çözülmemiş kalıyordu. Bu gözlem, saf OCR modellerinden genel amaçlı vision-language modellerine geçişin gerekçesi oldu.

10.3 Deneme 2 — Colab üzerinde VLM (Qwen3-VL)

Qwen3-VL-4B-Instruct tek istekte her iki katmanı da çözüyordu. Colab GPU’su üzerinde vLLM sunucusu olarak ayağa kaldırıldı. Bu aşamada iki teknik problem çözüldü:

FlashInfer/Blackwell (sm120) uyumsuzluğu: vLLM’in pip ile otomatik kurduğu FlashInfer wheel’i "FlashInfer requires GPUs with sm75 or higher" hatasıyla çöküyordu. Paketi kaldırmak çözüm olmadı (vLLM sampler’ı modül olarak arayıp ModuleNotFoundError verdi). Çözüm: vLLM’in kendi metadata’sından beklenen tam sürümü okuyup CUDA-uyumlu (cu129) wheel’e pinlemek ve VLLM_USE_FLASHINFER_SAMPLER=0 ile JIT sampling kernel’ini devre dışı bırakmak.

Çıktı formatı: JSON çıktı denendi ve 448 sayfada 91 parse hatası üretti — Türkçe metindeki tırnak ve özel karakterler JSON’u bozuyordu. Yerine satır-etiketli düz metin formatına geçildi ([CHAPTER]/[SUB]/[TEXT]/[TABLE]/[FRONTMATTER]): hem kırılmaz, hem de kısmen kesilen çıktılarda önceki satırlar kurtarılabilir.

Başarısızlık biçimi: ölçek. Model kalitesi yeterliydi, ancak Colab oturumları üzerinde binlerce kitabın işlenmesi kabul edilemez sürelerle ulaşıyordu (24 saatlik oturum limitleri, GPU tahsis dalgalanmaları, tek vLLM sunucusunun throughput tavanı). §5.3’teki BERT deneyiyle aynı desen: darboğaz doğruluk değil, ölçeklenebilirlikti.

10.4 Karar — API tabanlı batch işleme (Gemini 2.5 Flash-Lite)

Yerel GPU kısıtından çıkılıp bulut API’sine geçildi (gemini-2.5-flash-lite). Gerekçeler: Qwen3-VL için geliştirilen etiketli-satır promptu doğrudan taşınabiliyordu; Batch API sayfa başına senkron istek yerine toplu gönderim ve çok daha geniş bir rate-limit havuzu sağlıyordu; maliyet 1M token başına \$0.10 girdi / \$0.40 çıktı idi.

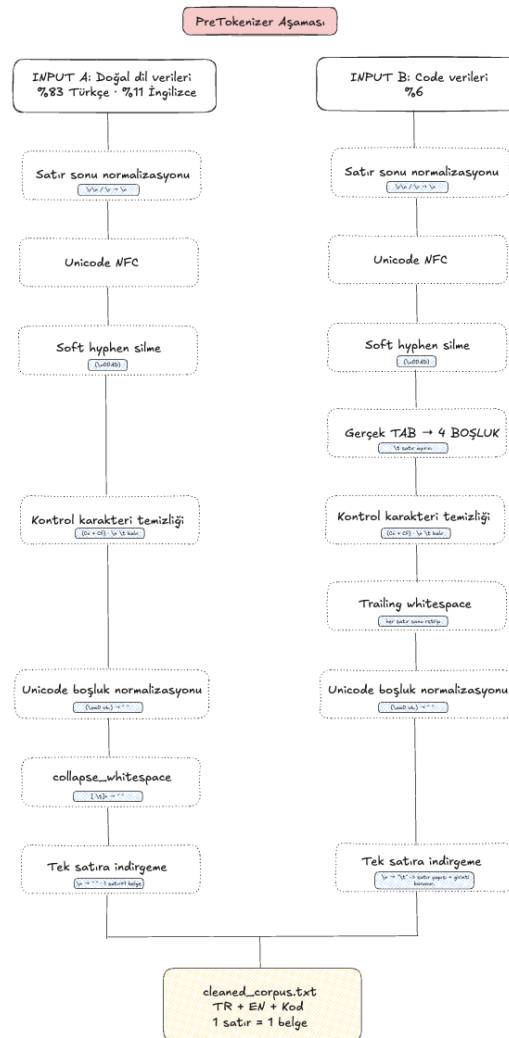
Prompt tasarımında bir ayrıntı, tekrarlanan bir hata sonrası eklendi: örnek çıktı bloğu bazı sayfalarda literal olarak kopyalanıyordu (model, örnekteki yer tutucu metni gerçek içerik sanıyordu). Prompt’a açık uyarı eklendi ve yer tutucular «...» ile işaretlendi.

Yapısal işleme tarafında iki mekanizma gerekti. Model her sayfayı bağımsız işlediği için aynı

bölüm başlığı birden fazla sayfada tekrar açılabilirdi (sayfa üst bilgisinde bölüm adı yazan kitaplarda sistematik olarak); ardışık ve normalize edilmiş aynı başlığa sahip chapter’lar otomatik birleştirildi. [FRONTMATTER] etiketli bloklar ise JSONL çıktısına hiç yazılmadı. Chunk’lama TARGET_CHUNK_CHARS = 6000 hedefiyle yapıldı (çok kısa subsection’lar birleştirilir, uzun olanlar 12.000 karakterde bölünür) — bu değer sonraki tokenizer eğitiminde max_sentence_length seçimini doğrudan etkiledi (bkz. §11.3).

11 Pre-tokenizasyon: Normalizasyon ve Kod Duyarlılığı

Tokenizer eğitiminden önce tüm kaynakların tek bir normalizasyon hattından geçirilmesi gerekiyordu. Literatür taraması (ModernBERT, Llama, Ettin tarifleri) ve mevcut Türkçe modellerin (BERTurk, TabiBERT, Mürşit) dokümantasyonu incelendiğinde iki bulgu hattın tasarımını belirledi: kod verisi düz metinden farklı normalizasyon gerektirir (girinti anlamlıdır) ve Unicode/bosluk işleme sessizce vocab israfına yol açar.



Şekil 2: Pre-Tokenizer aşaması: doğal dil (Türkçe/İngilizce) ve kod verileri için ayrı normalizasyon zincirleri.

Pre-Tokenizer aşaması: doğal dil verileri (%83 Türkçe · %11 İngilizce) ve kod verileri (%6) için

ayrı ama paralel normalizasyon zincirleri (satır sonu normalizasyonu, Unicode NFC, soft hyphen silme, kontrol karakteri temizliği, boşluk normalizasyonu), her ikisinin de cleaned_corpus.txt dosyasında (1 satır = 1 belge) birleşmesi.

11.1 Ortak hat

Türkçe ve İngilizce düz metin için uygulanan adımlar sırayla: satır sonu normalizasyonu (`\r\n`, `\r` → `\n`), Unicode NFC normalizasyonu, soft hyphen (U+00AD) temizliği, kontrol karakteri temizliği (Unicode Cc/Cf; `\n` ve `\t` hariç), Unicode boşluk normalizasyonu ve çoklu boşluk tekilleştirme (paragraf ayırıcı çift satır sonu korunarak).

Sıralama tesadüfi değildir. Satır sonu normalizasyonu her şeyden önce yapılmalıdır; aksi halde sonraki paragraf/başlık mantığı `\r`'yi ayrı karakter sayıp yanlış davranır.

Unicode boşluk normalizasyonunun gerekçesi ölçüme dayanır. HTML kaynaklarından kazınan metinde ` ` (U+00A0) çok yaygındı: mevzuat.gov.tr'de başlıklar bunlarla hizalanmıştı (bir örnekte 'Kanun Numarası' ile ':' arasında 26 ardışık), Yargıtay corpus'unda kayıtların %99'unda mevcuttu. Bu karakterler görünüşte normal boşluktur ama ayrı bir Unicode karakteridir: `[\t]+` regex'i onları görmez, kontrol karakteri temizliği de yakalamaz (o Cc/Cf'e bakar, bunlar Zs). Temizlenmezse tokenizer "Numarası" ile "Numarası\xa0" için ayrı token öğrenir. Bu adım çoklu boşluk tekilleştirmesinden önce gelmelidir.

Whitespace ve noktalama bölmesi bilinçli olarak elle uygulanmaz — SentencePiece'in kendi pre-tokenization'ına bırakılır. Elle bölme hem gereksiz tekrar hem tutarsızlık riskidir: Türkçe kesme işaretli kullanımların ("Ankara'da") nasıl ele alınacağına SentencePiece'in script-bazlı ayırma mantığı zaten karar verir.

11.2 Kod hattı

Kod kaynakları (C, C++, Go, Java, JavaScript, Python, TypeScript) için ayrı bir hat gerekti. Tasarım kısıtı: girinti korunur, satır sayısı sabit kalır.

Kısıt 1 — girinti silinirse anlam kaybolur. Python'da girinti syntax'ın parçasıdır. Ölçülen örnek: `collapse_whitespace` sonrası şu iki farklı program aynı token dizisine düşüyordu:

```
if x:
    y = 1
    z = 2

if x:
    y = 1
    z = 2    <-- bir seviye derin
```

Byte-level BPE kullanan TabiBERT ve Mürşit bu ikisini ayırt edebiliyordu, ilk hattımız edemiyordu.

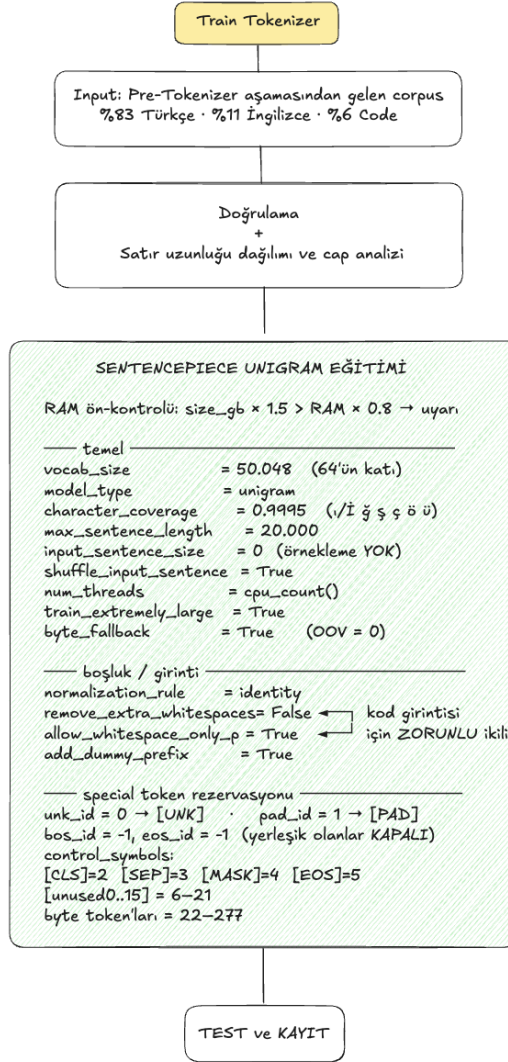
Kısıt 2 — kodun satır sonları olduğu gibi bırakılırsa corpus dengesi çöker. 521.000 kod dosyası ~15–52M dosya satırı eder; diğer tüm kaynakların toplamı ~900K satırdır. SentencePiece satır bazlı örneklediği için corpus'un %95'inden fazlası kod olurdu.

Çözüm: satır sonu → `\t` kaçıışı. Dosya satırını bölmez (satır sayısı belge sayısı kadar kalır), girinti boşluklarını bozmaz; gerçek tab'lar önce 4 boşluğa çevrilir. Bu çözümün zorunlu eşlik eden uyarı vardır: tokenizer `remove`

`allowbreakextra`

`allowbreakwhitespaces=False` ile eğitilmelidir, aksi halde SentencePiece normalizer'ı encode sırasında boşlukları yine siler ve corpus'taki girinti farkı boşa gider (ölçüldü).

Bir uyarı: \t kaçıışı yalnızca tokenizer eğitimi için bir tekniktir. Pretraining verisinde kod gerçek \n karakterleriyle saklanmalıdır — aksi halde model gerçek hayatta hiç karşılaşmayacağı sahte bir formatı öğrenir. JSONL formatında bu sorun kendiliğinden çözülür: satır sonları JSON string’i içinde escape edilir ve kaynak dengesi satır sayısıyla değil karışım ağırlıklarıyla kontrol edilir.



Şekil 3: Train Tokenizer akışı ve SentencePiece Unigram eğitim parametreleri.

Train Tokenizer akışı: pre-tokenizer çıktısının (%83 Türkçe · %11 İngilizce · %6 Code) doğrulanması ve satır uzunluğu/cap analizinin ardından SentencePiece Unigram eğitim kutusu (vocab_size, character_coverage, byte_fallback, boşluk/girinti ayarları, özel token rezervasyonu) ve test/kayıt adımı.

11.3 Tokenizer eğitiminde ölçülen üç problem

Problem	Belirti	Kök neden	Çözüm
Sessiz veri kaybı	Belgelerin ~%75'i kayboluyor	max_sentence_length varsayımı 4192 byte; daha uzun her satır sessizce atılır. Belgeler ~6KB hedefiyle chunk'lanmıştı	max_sentence_length=100000 — 266.639 satırdan yalnızca 2'si (OCR tekrar-döngüsü çöpi) atılıyor
Eğitim crash'i	Check failed: array.size() <= ... node_int_type	İç node index'i varsayılan 32-bit signed int (max ~2.15 milyar); corpus 1M cümleyle bile 3.63 milyar karaktere ulaştı. RAM ile ilgisi yok	train_extremely_large_corpus=True
Girinti çakışması	Farklı programlar aynı token dizisine düşüyor	Normalizer ardışık boşlukları siler; normalization_rule_name="identity" bunu kapatmaz (ayrı alan)	remove_extra_whitespace=False

11.4 Girinti token'ları: allow_whitespace_only_pieces

remove_extra_whitespace=False ile eğitilen ilk model girintileri koruyordu ama her boşluğu ayrı token olarak ürettiyordu. Ölçülen örnek (5 satırlık Python fonksiyonu, 82 karakter):

Model	Token	Davranış
remove_extra_whitespace=True	26	Girinti siliniyor — kayıplı
nowhitespace	46	Her boşluk ayrı token: 8 boşluk = 8 token
nowhitespace_test	30	Girinti tek token: <u> </u>

Kök neden vocab incelemesiyle bulundu: nowhitespace modelinin vocab'ında tek bir çoklu-boşluk token'ı yoktu. SentencePiece'te allow_whitespace_only_pieces varsayılan olarak False'tur ve “yalnızca boşluktan oluşan parça” öğrenilmesini yasaklar. remove_extra_whitespace=False boşlukları korur, ama trainer onları tek token'da birleştirmeye izinli olmadığı için her biri tek _ kalır.

Parametre eklendikten sonra model 3–16 boşluk arası 11 adet çoklu-boşluk token'ı öğrendi. Gerçek dosya üzerinde: 5.612 karakterlik Python dosyası 2.535 → 2.290 token (~%10 iyileşme), roundtrip decode(encode(x)) == x birebir doğru.

Gerçek kod corpus'unda (18 dil, 540 dosya) karşılaştırma:

Metrik	Eski (50k)	Final
Sıkıştırma (karakter/token)	2.783	2.722
Kayıpsız roundtrip oranı	%16	%100

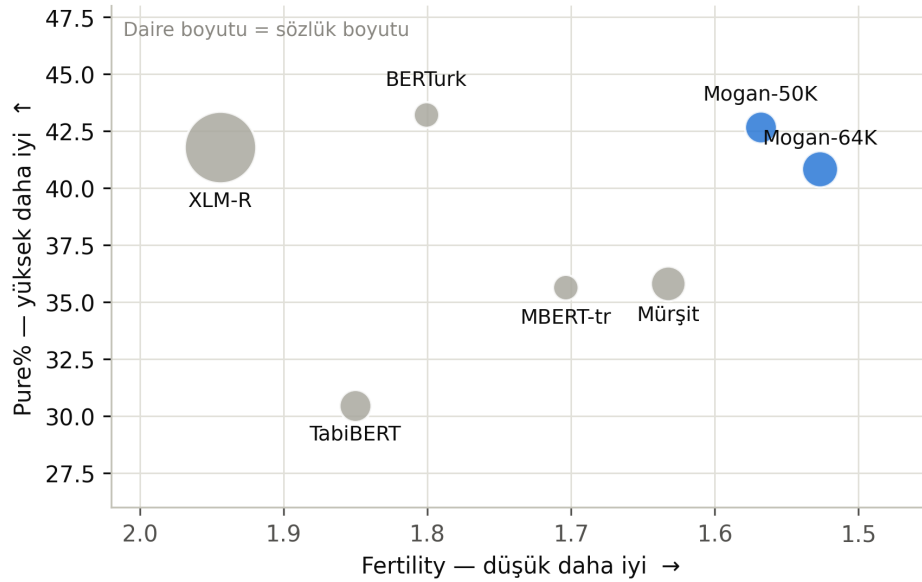
Roundtrip oranındaki sıçrama girinti korumasının doğrudan sonucudur: eski model kodun %84'ünü geri üretemiyordu. Rakip Türkçe tokenizer'larla aynı corpus üzerinde:

Tokenizer	Kod sıkıştırma	Kayıpsız
Mogan-FINAL	2.790	%100
Mürşit	2.454	%96
TabiBERT	2.196	%49
BERTurk	2.184	%0

12 Tokenizer Tasarımı: Vocab Boyutu ve Special Token'lar

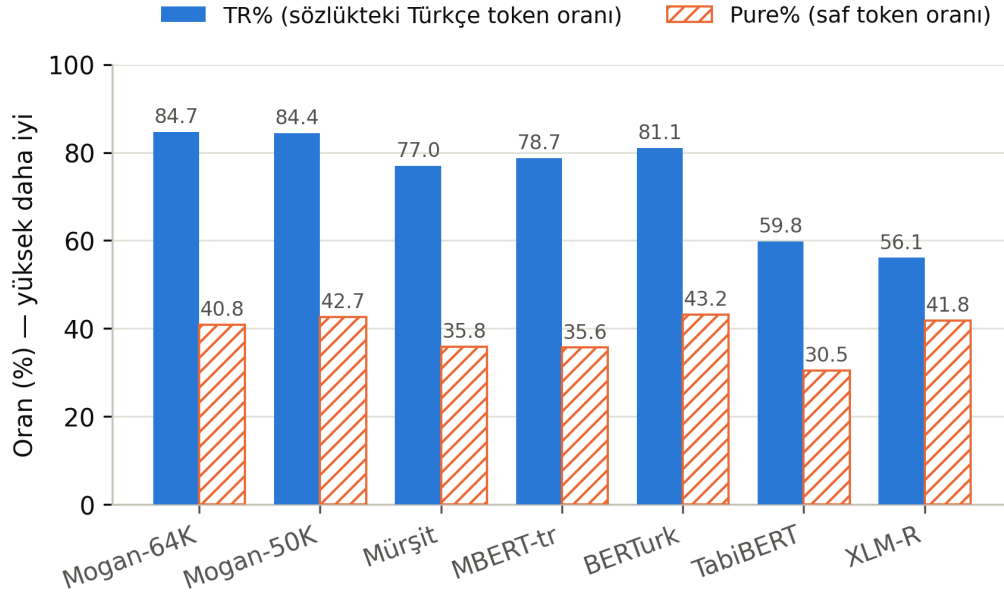
12.1 Vocab boyutu trade-off'u

Üç vocab boyutu eğitilip TR-MMLU (6.200 soru, 1.743.375 karakter) üzerinde ölçüldü. Verimlilik metrikleri multilingual-tokenizer-leaderboard tanımlarını izler (o leaderboard 13 dil destekler, Türkçe yoktur — metrikler kendi corpus'umuza uygulandı):



Şekil 4: Fertility (x, düşük daha iyi) vs Pure% (y, yüksek daha iyi); balon boyutu = vocab boyutu.

Dağılım grafiği: Fertility (x, düşük daha iyi) vs Pure% (y, yüksek daha iyi), balon boyutu = vocab boyutu. XLM-R, BERTurk, TabiBERT, Mürşit, MBERT-tr, Mogan-50K, Mogan-64K karşılaştırması.



Şekil 5: Tokenizer başına TR% (sözlükteki Türkçe token oranı) ve Pure% (saf token oranı) karşılaştırması.

Çubuk grafik: her tokenizer için TR% (sözlükteki Türkçe token oranı) ve Pure% (saf token oranı) yan yana.

Tokenizer	Vocab	Fertility ↓	Krk/Token ↑	TR% ↑	Pure% ↑
Mogan -64K	64.000	1.5268	5.0245	84.71	40.83
Mogan -50K	50.000	1.5682	4.8918	84.42	42.67
Mürşit	59.008	1.6326	4.6989	77.0	35.81
MBERT-tr	32.000	1.7037	4.5029	78.73	35.65
BERTürk	32.000	1.8008	4.26	81.06	43.22
TabiBERT	50.176	1.8501	4.1464	59.84	30.46
XLM-R	250.002	1.944	3.9462	56.09	41.78

Dilbilimsel metrikler (TR% = geçerli Türkçe benzersiz token oranı, Pure% = atomik token oranı) arXiv 2502.07057 tanımını izler; makale TR% ile MMLU skoru arasında $r=0.90$ korelasyon raporlar. Morfolojik analiz için Dilbaz (60.000+ kök, FST + kural motoru) kullanıldı.

64K vocab her verimlilik metriğinde 50K'yı geçiyor (fertility 1.527 vs 1.568). Ancak bu kazanç embedding parametresi maliyeti karşılığında gelir: 768 gizli boyutlu bir modelde 14.000 ek token $\approx 10.7M$ ek parametre — 150M'lık bir base model için parametre bütçesinin $\sim 7\%$ 'si. 2.7% 'lik fertility farkı bu maliyeti haklı çıkarmadığından 50K tabanı seçildi.

Nihai boyut 50048: 50.000'e en yakın 64'ün katı. GPU tensor çekirdekleri 64'ün katı boyutlarda daha verimli çalışır ve embedding matrisini ayrıca yuvarlama ihtiyacı kalkar.

Ek bağımsız doğrulama. TR-MMLU testine ek olarak, Mogan-64K, projenin kendi topladığı veriden tamamen bağımsız iki kaynaktan daha test edildi: Türkçe Wikipedia (wikimedia/wikipedia, 20231101.tr, 30.000 doküman) ve NLP literatüründe çok-dilli model/tokenizer değerlendirmesi için yaygın kullanılan akademik standart FLORES-200 (tur_Latn, dev+devtest, 2.009 cümle). Her iki testte de aynı seed (42) ile aynı doküman kümesi tüm tokenizer'lara uygulanmıştır.

Test seti	Tokenizer	Vocab	Char/Token ↑	Fertility ↓	UNK
Türkçe Wikipedia	Mogan -64K	64.000	4.705	1.640	0.00%
Türkçe Wikipedia	BERTurk	32.000	4.391	1.757	0.04%
Türkçe Wikipedia	Kumru-2B	50.176	4.010	1.924	0.00%
Türkçe Wikipedia	TabiBERT	50.176	4.010	1.924	0.00%
FLORES-200	Mogan -64K	64.000	5.277	1.460	0.00%
FLORES-200	BERTurk	32.000	4.932	1.563	0.002%
FLORES-200	TabiBERT	50.176	4.900	1.573	0.00%
FLORES-200	Kumru-2B	50.176	4.899	1.573	0.00%

Mogan-64K, her iki bağımsız test setinde de hem sıkıştırma (char/token) hem fertility metriğinde karşılaştırılan tüm tokenizer’ları geride bırakmıştır.

12.2 Special token yerleşimi

SentencePiece’te iki sembol listesi vardır ve ayırım kritiktir: control_symbols vocab’da bulunur ama metinden asla üretilmez; user_defined_symbols ise metinden üretilir. [MASK] mutlaka control_symbols olmalıdır — web verisinde literal olarak [MASK] yazan doküman bulunması olasıdır (NLP blog yazıları, dokümantasyon) ve user_defined_symbols kullanılırsa bu metin gerçek mask ID’sine çevrilerek eğitim verisine sahte mask enjekte edilir.

Nihai yerleşim: [UNK]=0, [PAD]=1, [CLS]=2, [SEP]=3, [MASK]=4, [EOS]=5, [unused0..15]=6–21, byte token’ları (byte_fallback=True) 22–277, öğrenilen parçalar 278+.

Yerleşik BOS/EOS mekanizması kapatıldı (bos_id=-1, eos_id=-1); aksi halde vocab’da aynı iş için iki takım token bulunurdu (<s>/</s> ve [CLS]/[EOS]) — hem slot israfı hem “dizi başı hangisi?” belirsizliği. [CLS] iki fazda da dizi başı rolünü üstlenir; CLM fazında öğrenilen embedding’i MLM fazına devrolur.

Reddedilen iki öneri. Türkçe ekleri user_defined_symbols olarak eklemek: bu liste metinde geçtiği her yerde atomik eşlenir, kelime sınırı tanımaz — lar eklenirse “Dilara”, “ıhlarmur” gibi köklerin ortasından zorla kesilir ve unigram’ın olasılık temelli optimal bölmesi devre dışı kalır; üstelik gereksizdir, unigram sık ekleri zaten kendiliğinden öğrenir. normalization_rule_name="nmt_nfkc": NFC zaten offline yapılıyor, ayrıca NFKC daha agresiftir (²→2, fi→fi) ve kod/matematik corpus’unda anlam taşıyan karakterleri bozar.

split_digits final eğitimde kullanılmadı: asıl getirisi üretken görevlerde ortaya çıkar, hedef mimari encoder olduğundan 2023’ün tek token olması sorun değildir (BERT ailesinin tamamı böyle çalışır).

12.3 HuggingFace dönüşümü ve bir parite hatası

Final model PreTrainedTokenizerFast’e çevrilirken iki karar önemlidir. Pre-tokenizer bilinçli olarak yoktur — Metaspace/Whitespace pre-tokenizer’ı kullanılsaydı ardışık _’lar tek tek bölünür ve allow_whitespace_only_pieces ile öğrenilen girinti token’ları encode’da asla seçilemezdi. Control parçalara -1000 skor verildi: SentencePiece proto’sunda bu parçalar skor 0.0 ile durur (log-olasılık 0 = en olası parça); SentencePiece runtime’ı CONTROL tipini metinden hiç eşleştirmedeği için sorun olmaz, ancak tokenizers.Unigram tip bilgisini tanımaz ve lattice metindeki literal

"[MASK]" yazısından ID 4 üretebilir (ölçüldü). Negatif skor bunu engeller; inference'taki atomik [MASK] eşleşmesi AddedToken mekanizmasından geldiği için etkilenmez.

Doğrulama: 10 farklı örnekte (Türkçe, kod, sayı, çoklu boşluk, emoji ve 5 gerçek corpus satırı) HF encode çıktısı ham SentencePiece çıktısıyla birebir aynı; kod roundtrip'i kayıpsız; korpus modunda literal [MASK] metni ID 4 üretmiyor.

13 Masking Stratejisi İçin GPU-suz Ön Analiz

MLM fazında hangi masking stratejisinin kullanılacağı (subword random / Whole Word Masking / morfolojik-farkında) bir eğitim koşusu harcanmadan corpus istatistikleriyle değerlendirildi. Beş domain'den (Hukuk, Makale, Book, CommonCrawl, FinePdfs) domain başına 3.000 belge örneklendi; ~15.9M kelime, 698.865 benzersiz tip. Örneklemede iki gürültü kaynağı filtrelendi: FinePdfs'te gömülü Çince/Arapça/Base64 blokları (ilk koşuda “en çok bölünen 15 kelime”nin tamamı bunlardı) ve biyoinformatik PDF'lerinden gelen saf ACGT dizileri.

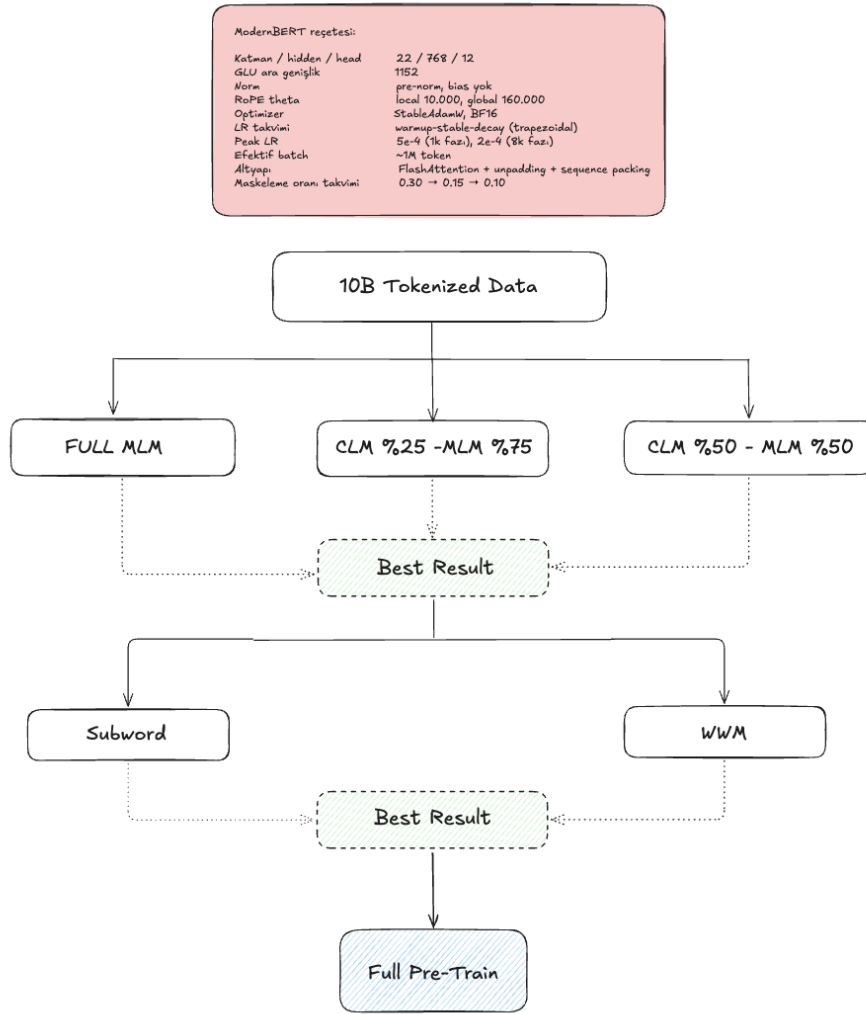
Metrik	Değer
Token/kelime — p50 / p90 / p99	2 / 4 / 6
Token-ağırlıklı ≥ 2 parça oranı	%41.1
Tip-ağırlıklı ≥ 2 parça oranı	%95.0
Kök-adayı / ek-adayı token payı	%76.6 / %23.4

Token-ağırlıklı ve tip-ağırlıklı oranlar arasındaki uçurum (%41 vs %95) metodolojik olarak belirleyicidir: kelime tiplerinin %95'i çok-parçalıdır ama bunlar nadir kelimelerdir; gerçek eğitim token akışının yalnızca %41'i çok-parçalı kelimelerden gelir.

Bu sayı doğrudan WWM'nin üst sınırıdır: token akışının %59'unda (tek-token kelimeler — “ve”, “bir”, “için” gibi yüksek frekanslı kelimeler) WWM ile token-level random masking zaten özdeştir. Ek-hedefli masking'in dokunabileceği pay ise %23.4'tür; agresif bir “sadece ekleri maskele” stratejisi modelin kök temsillerini eğitim sinyalinin dörtte birinden aزیyla besler.

Karar: İlk koşuda taban olarak token-level random masking (ABL-1), WWM ise ayrı bir ablasyon (ABL-2). WWM'nin uygulanması ucuzdur (– işaretinden kelime grubu çıkarımı) ve aşağı yönlü riski yoktur; ancak %41'lik tavan “kesin kazanç” değil “denemeye değer” beklentisi anlamına gelir. Tam morfolojik analiz (Zeyrek/Dilbaz ile offline etiketleme) yatırımı %23.4'lük bütçeyle gerekçelendirilemedi.

14 Pretraining Mimarisi: CLM→MLM Curriculum



Şekil 6: CLM→MLM curriculum karar akışı: ModernBERT reçetesinden Full Pre-Train’e giden yol.

ModernBERT reçetesi (katman/hidden/head, GLU ara genişlik, norm, RoPE theta, optimizer, LR takvimi, efektif batch, altyapı, maskeleme oranı takvimi) ve 10 milyar token’lık tokenize edilmiş veri üzerinde FULL MLM / CLM %25-MLM %75 / CLM %50-MLM %50 karşılaştırması ile Subword/WWM ablasyonuna, oradan Full Pre-Train’e giden karar akışı.

14.1 Literatür temeli

Encoder pretraining için standart yaklaşım saf MLM’dır. Ancak arXiv 2507.00994, 38 model (210M–1B) ve 15.000+ fine-tuning koşusuyla yapılan kontrollü ablasyonda sabit compute bütçesinde iki aşamalı CLM→MLM’in saf MLM’i tutarlı biçimde geçtiğini raporlar. 610M modelde sequence classification ortalaması: %100 MLM → 87.00; %25 CLM + %75 MLM → 87.85; %50/%50 → 87.58; %100 CLM → 83.58.

Mekanizma: CLM erken adımlarda daha veri-verimlidir (~5.000 adıma kadar MLM’i geçer) ve fine-tuning’de öğrenme oranına daha az duyarlıdır; MLM temsil kalitesinde sonunda öne

geçer. Kritik uygulama detayı: MLM fazı, LR decay’e girmemiş CLM checkpoint’inden devam etmelidir — WSD şedülünde geçiş “stable” fazın içinde yapılır, decay yalnızca en sonda gelir.

14.2 Mimari: dört sessiz-bug tuzacı

HuggingFace’in ModernBertModel’i bidirectional-only olduğundan attention yolu yeniden yazıldı. Dört tuzak, her biri sessizce yanlış model üretebilecek nitelikte:

1. Causal local pencere. ModernBERT alternating local/global attention kullanır ve local pencere simetrik (w/2 sol + w/2 sağ). CLM fazında simetrik pencere geleceği sızdırır — ve bu loss’ta fark edilmez, model sessizce “kopya çeker”. Pencere causal kipte (w-1, 0) olmalı, bidirectional kipte (w/2, w/2)’ye dönmelidir.
2. Boundary-aware packing. Dokümanlar sabit bloklara paketlenir ama attention doküman sınırını geçmemelidir (cu_seqlens).
3. Position ID reset. Paketlenmiş dizide RoPE pozisyonları her doküman için sıfırdan başlamalıdır; aksi halde paketin sonundaki kısa doküman, hiç görmediği 900 token’lık bir geçmişin pozisyon kodlamasını alır. Bu projede pozisyonlar tek kaynaktan (collator’ın açık position_ids’i) gelir; FlashAttention’ın kendi iç rotary’si bilinçli devre dışıdır ki pozisyonlar doğrulanabilir olsun.
4. Loss’ta doküman sınırı maskesi. cu_seqlens attention’ı keser ama loss’u kesmez: shifted-label CLM loss’unda paketteki i. dokümanın son token’ı, (i+1). dokümanın ilk token’ını tahmin etmeye çalışır. Her dokümanın son pozisyonunun etiketi -100 olmalıdır.

14.3 Ablation için bellek ve sayısal kararlar

Chunk’lı ve gradient-checkpointed cross-entropy. CLM fazında pozisyonların ~%99’u etiketlidir. ~98.000 token’lık bir batch’te logits tensörü tek başına $98K \times 50.048$ bf16 ≈ 9.8 GB, .float() ile +19.5 GB, log_softmax ile +19.5 GB eder — 96GB’lık kartta bile geçmez. Düz chunk’lama yetmez: autograd her chunk’ın logits’ini backward için saklar. torch.utils.checkpoint ile logits backward sırasında yeniden hesaplanır ve tepe bellek tek chunk’a (~1.6 GB) iner. Eşdeğerlik testle doğrulandı (fark 0.00e+00).

14.4 Ablation Veri hattı

Kaynak	Token
CommonCrawl	3.63B
Code	1.70B
FineWeb-edu-eng	1.25B
Hukuk	780M
Makale	627M
FinePdfs	588M
Book	316M
FineMath-4Plus	303M
turkce-matematik-data	221M
Toplam	~9.4B

15 Ablasyon Sonuçları: CLM→MLM Curriculum vs. Saf MLM

§14.1’de literatürden aktarılan iddia — sabit compute bütçesinde iki aşamalı CLM→MLM’in saf MLM’i geçtiği — kendi mimarımız, kendi tokenizer’ımız (50.048 vocab) ve kendi Türkçe corpus’umuz üzerinde doğrudan sınandı. Bu bölüm iki ablasyon koşusunun 10.000. adımdaki checkpoint’lerinin karşılaştırmasını raporlar.

15.1 Deney kurulumu

İki koşu da aynı veri hattı, aynı mimari ve aynı toplam adım sayısı (10.000) ile eğitildi:

- **ABL-1 (mlm_only):** 10.000 adım saf MLM.
- **ABL-2 (clm25_mlm75):** ilk %25 CLM (~2.500 adım), kalan %75 MLM (~7.500 adım); geçiş, WSD şedülünün “stable” fazı içinde, LR decay’e girilmeden yapıldı.

Değerlendirme protokolü her iki checkpoint için birebir aynıdır: seed=1234, mask_rate=0.30, max_len=512 (eğitim block_size=1024 ile uyumlu), torch 2.11.0+cu128, transformers 4.57.6. Linear probe’lar aynı “paket” bölmesiyle ve aynı örnek sayılarıyla koşturuldu; MS MARCO tarafında n_query=5000, pool_size=5000, doc_trunc=2000 sabittir. Karşılaştırma, her iki koşuda da aynı protokolle ölçülmüş iki dış kaynaklı metrik ailesi üzerinden yapılır: dört linear probe görevi (temsil kalitesi) ve MS MARCO retrieval (anlamsal ayrıştırma gücü).

Sağlık kontrolleri her iki modelde de temizdir: bidirectionality testi (sağdaki token değiştirildiğinde tahmin logit’inin değişmesi) ABL-1’de 10.81, ABL-2’de 9.75 maksimum logit farkı verdi; tokenizer’ın [MASK] ID’si (4) eğitimde kullanılan ID ile eşleşiyor; eval uzunluğu eğitim blok boyutuyla uyumlu.

Önemli asimetri. Eşit toplam adımda ABL-1, ABL-2’ye göre %33 daha fazla MLM adımı görmüştür (10.000 vs 7.500). Aşağıdaki farklar bu handikaba *rağmen* curriculum lehine elde edilmiştir; yani ölçülen fark, eşit-MLM-adımı kurgusunda beklenenden muhafazakârdır.

15.2 Linear probe: temsil kalitesi

Probe’lar donuk (frozen) temsiller üzerine eğitilen tek katmanlı doğrusal sınıflandırıcılardır; modelin temsilde bilginin *doğrusal olarak ayrılabilir* biçimde bulunup bulunmadığını ölçer.

Tablo 1: Linear probe doğrulukları (%). Aynı split, aynı örnek sayıları.

Görev	Sınıf	Majority	MLM-only	CLM25+MLM75	Δ
POS (BOUN), n=7.990	15	29.85	87.62	87.42	-0.20
Haber kategorisi, n=230	5	20.00	97.39	97.39	0.00
NLI, n=2.001	3	33.38	48.53	50.22	+1.69
Ürün yorumu (dengeli), n=2.000	2	50.00	80.75	81.80	+1.05
Ortalama	—	—	78.57	79.21	+0.64

Okuma: probe tarafında iki model pratik olarak *eşittir*. Ortalamadaki +0.64 puanlık fark, tek seed ve tek checkpoint ile ölçülmüş olduğundan koşu-içi gürültüden ayrıştırılamaz. Görev bazında:

- **POS ve haber kategorisi ayırt edici değildir.** POS'ta fark 0.20 puandır (7.990 örnekte ~16 tahmin); haber kategorisinde iki model 230 örnekte *aynı* doğruluğu (%97.39) veriyor — görev doymuş, ölçüm çözümlülüğü kalmamış.
- **Fark, semantik cümle-gifti görevlerinde toplanıyor.** Tek anlamlı sinyal NLI (+1.69) ve ürün yorumu (+1.05) üzerindedir; ikisi de tekil token etiketi değil, cümle/belge düzeyi anlam temsili gerektirir. Bu, §15.3'teki retrieval bulgusuyla aynı yönü gösterir.
- **NLI her iki modelde de zayıftır** (majority %33.38'e karşı %48–50). 10.000 adımlık bir ablasyon checkpoint'i için beklenen bir taban seviyesidir; mimari bir arıza işareti değildir, ancak NLI'nın full pre-train sonrası yeniden ölçülmesi gerekir.

Bu +0.64 puanlık ortalama fark, arXiv 2507.00994'ün 610M modelde sequence classification ortalamasında raporladığı farkla (%100 MLM = 87.00 vs %25 CLM + %75 MLM = 87.85; +0.85) hem yön hem büyüklük mertebesi olarak tutarlıdır.

15.3 MS MARCO: asıl ayrışma

Aynı checkpoint'ler contrastive fine-tuning olmadan, doğrudan pretraining temsilleriyle Türkçe MS MARCO üzerinde retrieval'a sokuldu (5.000 sorgu, 5.000 belgelik havuz). Bu, ablasyondaki tek doğrudan Türkçe anlamsal erişim ölçümüdür. Contrastive bir hedefle eğitim yapılmadığı için mutlak değerler düşüktür; anlamlı olan iki model arasındaki *orandır*.

Tablo 2: MS MARCO retrieval (%). n_query=5000, pool=5000, doc_trunc=2000, rastgele taban @1 = 0.02.

Metrik	MLM-only	CLM25+MLM75	Δ	Oran
mean R@1	2.96	10.86	+7.90	×3.67
mean R@5	7.28	20.84	+13.56	×2.86
mean R@10	9.92	26.34	+16.42	×2.66
mean MRR	5.59	16.23	+10.64	×2.90
CLS R@1	0.42	2.90	+2.48	×6.90
CLS R@10	2.00	6.42	+4.42	×3.21
CLS MRR	1.07	4.25	+3.18	×3.97

Bulgular:

1. **Curriculum modeli her metrikte 2.7–6.9 kat öndedir.** Rastgele tabana (%0.02) göre normalize edildiğinde ABL-2 ~543×, ABL-1 ~148× taban seviyesindedir. Bu, probe'lardaki 1 puanlık farklardan niteliksel olarak farklı bir ayrışmadır ve tek-seed gürültüsüyle açıklanamaz.
2. **Fark, ABL-1'in daha fazla MLM adımı görmesine rağmen ortaya çıkmıştır** (§15.1). CLM fazına ayrılan %25'lik bütçe, MLM'den çalınmış olmasına karşın net kazanç üretiyor.
3. **CLS pooling her iki modelde de kullanılamaz durumdadır.** CLS %2.90 ile mean pooling %10.86 arasındaki 3.7 katlık uçurum beklenen bir sonuçtur: [CLS] embedding'i NSP/contrastive gibi bir cümle düzeyi hedefle hiç eğitilmemiştir, dolayısıyla dizi özeti

taşımaz. **Downstream retrieval kullanımında mean pooling varsayılan olmalıdır;** CLS ancak ayrıca bir cümle-düzeyi hedefle eğitilirse anlamlıdır.

4. **Mutlak değerler yayınlanmış MS MARCO skorlarıyla kıyaslanamaz.** Bura-daki model bir retriever değil, ham bir pretraining encoder'ıdır: contrastive eğitim, query/document ayrımı ve hard-negative örnekleme yoktur; belgeler 2.000 karakterde kesilmiştir. Tablo, mimari kararı için bir *sıralama* ölçütüdür, erişim kalitesinin mutlak bir göstergesi değildir.

15.4 Neden: embedding geometrisi

Probe'ların eşit, retrieval'ın 3 kat farklı çıkması çelişki değildir; iki metrik farklı şeye duyarlıdır. Probe öğrenilen bir doğrusal dönüşümdür ve bilgi taşımayan baskın yönleri yeniden ölçekleyip yok sayabilir; cosine retrieval bunu yapamaz, ham geometriye doğrudan bağlıdır.

Tablo 3: Embedding sağlığı ve STS (1.000 örnek; STS n=1.379).

Metrik	MLM-only	CLM25+MLM75
cos_raw (anizotropi)	0.9731	0.9595
Efektif rank	70.18	55.59
eff_rank_ratio	0.0914	0.0724
1. bileşenin varyans payı	0.2810	0.1187
İlk 10 bileşenin varyans payı	0.4896	0.5675
STS Spearman	0.4833	0.5026
STS Pearson	0.4536	0.4905

Kritik satır 1. bileşenin varyans payıdır: ABL-1'de tek bir yön varyansın %28.1'ini yutuyor, ABL-2'de bu oran %11.9. Literatürde “rogue dimension” olarak bilinen bu desen tüm vektörleri ortak bir yöne doğru sıkıştırır — cos_raw farkı (0.9731 vs 0.9595) da bunu doğrular. Cosine benzerliği bu ortak bileşeni ayıklayamadığından ayırt edici sinyal gürültüye gömülür. ABL-1'in *daha yüksek* efektif rank'a (70.18 vs 55.59) sahip olmasına rağmen retrieval'da 3 kat geride kalması bu yüzdendir: efektif rank tek başına downstream performansı öngörmez; varyansın ilk bileşenler arasında nasıl dağıldığı belirleyicidir. STS'teki tutarlı fark (+0.019 Spearman, +0.037 Pearson) — ki o da cosine tabanlıdır — aynı mekanizmanın bağımsız bir doğrulamasıdır.

Not: her iki model de “efektif rank düşük, izlenmeli” uyarısı almıştır (eff_rank_ratio %7–9). 10.000 adımlık checkpoint'ler için beklenen bir durumdur, ancak full pre-train boyunca bu metrik takip edilmelidir.

15.5 Sonuç ve sınırlılıklar

Karar: full pre-train, **ABL-2 (CLM %25 → MLM %75)** konfigürasyonu ile yapılacaktır. Gerekçe: probe metriklerinde en kötü ihtimalle eşitlik (ortalama +0.64), buna karşılık cosine tabanlı tüm görevlerde (MS MARCO 2.7–6.9×, STS +0.019 Spearman) net ve tek yönlü üstünlük — üstelik %33 daha az MLM adımıyla. §13'te tanımlanan WWM ablasyonu bu taban üzerine kurulacaktır.

Sınırlılıklar:

- Tek seed, tek adım noktası (10.000); hata payı ölçülmedi. Probe farkları (≤ 1.7 puan) bu nedenle “eşitlik”, MS MARCO farkları “ayrışma” olarak okunmalıdır.
- Probe görevlerinin ikisi (POS, haber kategorisi) doymuş durumdadır ve modeller arası ayırım üretmemektedir; sonraki ablasyonlar için daha zorlayıcı görevler gerekir.
- Retrieval ayağı tek bir veri kümesine (Türkçe MS MARCO) dayanır; bulgunun farklı alan ve sorgu tiplerinde de geçerli olduğu ayrıca doğrulanmalıdır.
- Sonuçlar 10.000 adımlık kısa koşulara aittir; curriculum’un avantajının full pre-train ölçeğinde korunup korunmadığı ayrıca doğrulanmalıdır.